Ders 7 Genişletilmiş Notu: PPM (Escape, PPMA/PPMB/PPMC, Trie), Sözlük Tabanlı Yöntemlere Giriş ve String Sıkıştırma

Ders 7 Genişletilmiş Notu: PPM (Escape, PPMA/PPMB/PPMC, Trie), Sözlük Tabanlı Yöntemlere Giriş ve String Sıkıştırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. PPM'e Dönüş: Order-N Markov Modeli ve Context Tablosu
- 2. PPM'in Çalışma Prensibi: Switching
- 3. Kod Çözücü ve Escape Sembolü
- 4. "assanissimassa" Örneğinde Escape'li Tablo
- 5. Dört Senaryo: Bir Sembolü Kodlamak İçin Kaç Bit Gerekir?
- 6. Yeni Sembolün Tabloya Eklenmesi ve Kod Çözme
- 7. PPM'in Versiyonları: PPMA, PPMB, PPMC
- 8. Pratik Gerçekleştirme: Trie Ağaçları
- 9. Vine Pointer ile Switch Etme
- 10. Ders İçi Alıştırma: Huffman Kodları ve Aritmetik Kodlama Tablosu
- 11. Sözlük Tabanlı Yöntemlere Giriş
- 12. Statik Sözlükle Kodlama ve Sözlükte Bulunma Oranı
- 13. String Sıkıştırma: Neden Karakter Değil String?
- Örnek Sorular

1. PPM'e Dönüş: Order-N Markov Modeli ve Context Tablosu

Önceki derste PPM (Prediction with Partial String Matching) anlatılmaya başlanmıştı. Hoca, aritmetik kodlamanın formüllerinin aslında çok kolay olduğunu hatırlattı. Ondalık (float) sayılar yerine tamsayılarla çalışan ve daha hızlı olduğu söylenen bir sürümü de vardır; hoca bu dersi açarken kendi yazdığı kodlarda yalnız ondalık sürümü kodladığını, onun da çok zor olmadığını söyledi.

PPM'de en önemli şey Order-N Markov modelidir. Bir sembole olasılık verilirken yalnız o sembolün doküman içinde karşımıza gelme olasılığı kullanılmaz; sembolün geçmişindeki N sembole bakılarak, o geçmişle birlikte gelme olasılığı kullanılır. Örneğin "PR" ardından gelen "E" ile "TT" ardından gelen "E" farklı olasılıklara sahiptir.

Hoca önce $N = 3$ için "sembolün kendisi de dahil" dedi, sonra kendini düzeltti: context yalnız geçmişi verir, sembolün kendisi context'e dahil değildir. Order 4'te önceki 4 sembol, Order 3'te 3, Order 2'de 2, Order 1'de 1 sembol context'tir; sembol ilk kez geldiğinde de Order 0 tablosuna girer.

1.1 İki aşamalı çalışma

Algoritma iki aşamada çalışır:

- Bütün metin okunarak context tablosu oluşturulur ve her context için olasılık değerleri bulunur.
- Bu olasılık değerleri bir istatistiksel kodlayıcıya, burada aritmetik kodlamaya gönderilir. Kod çözme de aritmetik kodlamanın ters işlemiyle yapılır; elde edilen değerin tablodaki hangi sembole karşılık geldiğine bakılır.

  Hocanın vurgusu: Bu tabloyu oluşturmak çok önemlidir; doğru ve net olarak oluşturulması gerekir.

1.2 Tablonun artımlı güncellenmesi (ekrandan, 00:18:30)

Slayttaki tablo, "xyzzxyxyzzx" metni için Order 4'ten Order 0'a kadar context'leri ve sayılarını gösterir (Salomon, Table 2.72):

```
(a) "xyzzxyxyzzx"
Order 4        Order 3       Order 2      Order 1     Order 0
xyzz->x 2      xyz->z 2      xy->z 2      x->y 3      x 4
yzzx->y 1      yzz->x 2        ->x 1      y->z 2      y 3
zzxy->x 1      zzx->y 1      yz->z 2       ->x 1      z 4
zxyx->y 1      zxy->x 1      zz->x 2      z->z 2
xyxy->z 1      xyx->y 1      zx->y 1       ->x 2
yxyz->z 1      yxy->z 1      yx->y 1
```

Bir "z" daha gelince tablo (b) güncellenir: Order 0'da z 5 olur; Order 1'de x'i izleyen z (x→z 1) eklenir; Order 2'de xy→z 1, Order 3'te zzx→z 1, Order 4'te yzzx→z 1 eklenir.

Hoca ekranda bunu kendi küçük örneğiyle gösterdi: metnin sonunda "xxxy" olsun ve "y" gelmiş olsun. Önce Order 0'da y'nin frekansı 3'ten 4'e çıkarılır. Sonra Order 1'e bakılır: y'den önce x gelen bir context (x→y) var mı? Varsa sayısı artırılır. Sonra Order 2: iki x'ten sonra gelen y (xx→y) var mı? Yoksa 1 sayısıyla eklenir. Order 3'te de xxx→y yoksa 1 ile eklenir. Bütün doküman, seçilen order modeline göre bu şekilde işlenir.

2. PPM'in Çalışma Prensibi: Switching

Slayttaki ilkeler şunlardır (ekrandan, 00:20:00):

- PPM, sıfır olasılıklı bir context geldiğinde onu daha kısa uzunluktaki bir context ile yer değiştirir (switching).
- PPM, order-N context ile başlar. Gelen bir S sembolünü order-N modeline göre önceden gelmiş olan C context'i içinde arar.
- Bulamazsa (S sembolünün izlediği bir C context'i mevcut değilse) Order-$(N-1)$'e switch eder.
- PPM kodlayıcısı Order-0 context'ine switch ettiğinde S sembolünün geçmişte kaç kez göründüğüne bakar; daha önce hiç görülmemişse S için sabit bir olasılık değeri atanır: $1/$alfabenin boyu.
- Bulunan P olasılıklı her S sembolü kodlanmak için aritmetik kodlama algoritmasına gönderilir.

Hoca bunu bir örnekle açıkladı. Tablo bir dokümandan çıkarılmış olsun ve aynı bilgilerle başka bir doküman kodlanmak istensin. Yeni dokümanda "pqyz" ardından "z" gelsin. Önce bu dörtlü context aranır; tabloda yoksa order bir azaltılır ve "qyz" context'i aranır. O da yoksa yine switch edilir ve "yz" aranır. "yz" varsa onun olasılık değeri kullanılarak işleme devam edilir. Yani her zaman en uzun pattern alınır; olasılığı sıfırsa, yani daha önce karşımıza çıkmamışsa, bir aşağıya inilir.

Order 4'ten başlayıp 3, 2, 1 ve 0'da da bulunamayan sembol hiç görülmemiş bir semboldür. Bunun olasılığı alfabedeki sembol sayısına göre verilir; hoca Türkçe için 1/29, İngilizce için 1/26 örneğini verdi. Boşluk ve özel karakterler de eklenirse alfabe daha da büyür.

Aritmetik kodlamada karakterler ve olasılıkları nasıl kodlayıcıya gönderiliyorsa, PPM'de de çıkarılan olasılık değerleri aritmetik kodlamaya gönderilir. Aritmetik kodlama bütün metne 0 ile 1 arasında bir değer atar; kod çözerken aynı olasılık değerleriyle geri adımlar yapılarak orijinal metne dönülür. Tek fark, PPM'de olasılığın tek bir karaktere değil, N değerine göre 2, 3, 4 karakterlik bir bütüne verilmesidir.

3. Kod Çözücü ve Escape Sembolü

PPM kod çözücü, kodu çözerken hangi aşamada switching işleminin gerçekleştiğini bilmek zorundadır. Bunu bildiren sembole **escape** sembolü denir (ekrandan, 00:20:00):

- Escape sembolünün de bir olasılık değeri vardır.
- Kodlayıcı, switch işlemini gerçekleştirmeden önce escape sembolünün olasılığını aritmetik kodlamaya gönderir; escape her switch işleminde bir sembolmüş gibi gönderilir.

İlk derste kurulan tabloda escape yoktu; bu derste tabloya escape eklenir.

4. “assanissimassa” Örneğinde Escape’li Tablo

Hoca, kısa ve tekrar eden kelimelerden oluşan bir metin seçildiğini söyledi: “assanissimassa” (ekranda “as sanis sima ssa” diye bölünmüş). Model Order 2’dir. Tabloda her context grubundan sonra bir escape satırı vardır (ekrandan, 00:30:00):

```
Order 2            Order 1            Order 0
Context f  p       Context f  p       Symbol f  p
as->s  2  2/3      a->s   2  2/5       a      4  4/19
esc    1  1/3      a->n   1  1/5       s      6  6/19
                   esc    2  2/5       n      1  1/19
ss->a  2  2/5                          i      2  2/19
ss->i  1  1/5      s->s   3  3/9       m      1  1/19
esc    2  2/5      s->a   2  2/9       esc    5  5/19
                   s->i   1  1/9
sa->n  1  1/2      esc    3  3/9
esc    1  1/2
                   n->i   1  1/2
an->i  1  1/2      esc    1  1/2
esc    1  1/2
                   i->s   1  1/4
ni->s  1  1/2      i->m   1  1/4
esc    1  1/2      esc    2  2/4

is->s  1  1/2      m->a   1  1/2
esc    1  1/2      esc    1  1/2

si->m  1  1/2
esc    1  1/2

im->a  1  1/2
esc    1  1/2

ma->s  1  1/2
esc    1  1/2
```

Burada f sembolün o context’te kaç kez karşımıza çıktığı (frekans), p olasılıktır.

4.1 Tablonun adım adım kurulması

Hoca tabloyu metni soldan sağa okuyarak kurdu:

- **a:** Order 0’da a = 1. Geçmişi yok.
- **s:** Order 0’da s = 1. Geçmişinde a var: a→s = 1.
- **s:** Order 0’da s = 2. s→s = 1; geçmişi “as” olduğu için as→s = 1.
- **a:** a = 2. s→a = 1, ss→a = 1.
- **n:** n = 1. a→n = 1, sa→n = 1.
- **i:** i = 1. n→i = 1, an→i = 1.
- **s:** s = 3. i→s = 1, ni→s = 1.
- **s:** s artar. s→s = 2, is→s = 1.
- **i:** ss→i ve s→i eklenir.

- **m:** m = 1. i→m = 1, si→m = 1.
- **a:** m→a = 1, im→a = 1.
- **s, s, a:** s sayısı 6'ya, a sayısı 4'e çıkar; a→s 2, ma→s 1, s→s 3, as→s 2, s→a 2, ss→a 2 olur.

Hoca sayarken bir yerde bir s'yi atladığını fark etti; slayttaki sayılar doğrudur. Tablonun bütün sayıları python ile metinden yeniden çıkarıldı ve slaytla aynı bulundu.

4.2 Escape frekansının bulunması

Order 0'da escape neden 5'tir? Escape dışında 5 farklı sembol (a, s, n, i, m) vardır; bu yüzden escape'e 5 atanır. Toplam neden 19'dur? a 4, s 6 (toplam 10), n 1 (11), i 2 (13), m 1 (14) ve escape 5 ile 19. Bu yüzden a'nın olasılığı 4/19, s'nin 6/19'dur.

Order 1 ve Order 2'de de her context'in kendi escape'i vardır. Escape'e atanan frekans, o context'i izleyen farklı sembol sayısıdır:

- **a→** context'i: s ve n olmak üzere 2 farklı sembol izler; escape 2. Toplam frekans 2 + 1 + 2 = 5; olasılıklar 2/5, 1/5, 2/5.
- **s→** context'i: s, a, i olmak üzere 3 farklı sembol; escape 3. Toplam 3 + 2 + 1 + 3 = 9; olasılıklar 3/9, 2/9, 1/9, 3/9.
- **n→** context'i: yalnız i; escape 1. Toplam 2; olasılıklar 1/2, 1/2.

Hoca, escape eklemenin hiçbir zorluk getirmediğini söyledi: yalnız her context için farklı sembol sayısı kadar bir escape frekansı eklenir.

> **Ek bilgi (derste anlatılmadı):** Escape frekansını context'i izleyen farklı sembol sayısı kadar almak, büyük olasılıkla PPMC olarak bilinen yöntemin kuralıdır. Hoca da 7. bölümde derste gösterilen yöntemin PPMC olduğunu söyledi.

5. Dört Senaryo: Bir Sembolü Kodlamak İçin Kaç Bit Gerekir?

Girdi "...ssa" ile bitmişti. Son context Order 2 için "sa" olsun ve sıradaki sembol için dört senaryo düşünelim. Bir olasılığın 2 tabanındaki logaritmasının eksisi, $-\log_2 p$, o sembolü kodlamak için gereken bit sayısını verir.

5.1 Senaryo 1: "n" gelir (ekrandan, 00:35:40)

Önce Order 2'ye bakılır: "sa" context'ini izleyen bir "n" var mı? Vardır (sa→n) ve olasılığı 1/2'dir. "n" bu olasılıkla aritmetik kodlamaya gönderilir:

$$-\log_2(1/2) = 1 \text{ bit}$$

Hoca, aritmetik kodlamaya yalnız n'nin olasılığının gönderildiğini, ama bu olasılığın Markov modeline göre context'e bakılarak bulunduğunu vurguladı.

5.2 Senaryo 2: "s" gelir

Order 2'de "sa"yı izleyen bir "s" yoktur. Switch edilmesi gerekir; karşı tarafa bunu bildirmek için bu context'in escape'i (1/2) gönderilir. Order 1'e geçilir: "a"yı izleyen "s" var mı? Vardır, olasılığı 2/5'tir.

- Escape: $-\log_2(1/2) = 1$ bit.
- a→s: $-\log_2(2/5) \approx 1{,}32$ bit.
- Toplam: $1 + 1{,}32 = 2{,}32$ bit.

5.3 Senaryo 3: "m" gelir (ekrandan, 00:40:00)

Slayttaki adımlar:

- "m"nin izlediği "sa" context'i aranır (order-2). Bulunamayınca 1 bitlik escape gönderilir ve Order-1'e switch edilir.

- "m"nin izlediği "a" context'i order-1'de aranır. Bulunamayınca $-\log_2(2/5) = 1{,}32$ bitlik escape gönderilir ve Order-0'a switch edilir.
- "m"nin olasılığı 1/19 bulunarak $-\log_2(1/19) = 4{,}25$ bit ile kodlanır.
- Toplam $1 + 1{,}32 + 4{,}25 = 6{,}57$ bit.

Hoca, bunun bile normal kodlamadan iyi olduğunu söyledi: ASCII ile bu sembol 8 bitle kodlanırdı. Uygun context'lerle gelen bir sembol belki 2 bitle, belki 1 bitle kodlanır; bu tamamen metnin içeriğine bağlıdır.

5.4 Senaryo 4: "d" gelir (ekrandan, 00:42:20)

"sa"yı izleyen "d" yoktur; escape 1/2. "a"yı izleyen "d" de yoktur; escape 2/5. Order 0'da da "d" yoktur; yani daha önce hiç görülmemiş bir sembol gelmiştir. Order 0'ın escape'i (5/19) gönderilir ve Order -1'e geçilir. Burada "d"ye sabit olasılık, $1/$alfabenin boyu atanır; slaytta alfabe 28 sembol alınmıştır.

```
order-2   -log2(1/2)  = 1    bit
order-1   -log2(2/5)  = 1.32 bit
order-0   -log2(5/19) = 1.93 bit
order--1  "d" 1/28 olasılıkla  -log2(1/28) = 4.8 bit
Toplam 9.05 bit gerekir
```

Değerler python ile doğrulandı: $-\log_2(5/19) \approx 1{,}926$, $-\log_2(1/28) \approx 4{,}807$, toplam $\approx 9{,}055$ bit.

> **Ders sonrası not:** Hoca bu örnekte Order-0 escape'i için "5 bölü 9" dedi; slayttaki ve tablodaki doğru değer 5/19'dur. 3. senaryoda "1 bölü 32" dediği yer de 1,32 bittir.

6. Yeni Sembolün Tabloya Eklenmesi ve Kod Çözme

Soru: Hocam, burada neden order kullanmadın? **Cevap:** Hoca bu soruyu kendisi sorup cevapladı: "d" bir kez karşımıza geldiği anda hemen tabloya eklenir. Order 0'da d = 1 olur, escape 6'ya, toplam 20'ye çıkar ve d'nin olasılığı 1/20 olur. Yeni gelen "d"nin Order 1 ve Order 2 context'lerine de bakılır ve gerekiyorsa yeni değerler eklenir. Böylece bir sonraki "d" geldiğinde daha büyük bir olasılıkla, yani daha az bitle kodlanır.

Aslında tablo başlangıçta da böyle oluşur: "a" ilk kez geldiğinde Order -1'de durur, $1/$alfabe boyu ile kodlanır; sonra hemen Order 0'a eklenir. Ardından geçmişine bakılır ve "ad" gibi context'ler eklenir. "sa" geçmişiyle bir "d" daha önce çıkmış olamaz, çünkü "d" ilk kez gelmiştir; bu context de bir kez görülmüş olarak eklenir.

Kod çözmede aritmetik kodlamanın aralıkları açılarak değerler elde edilir ve bu değerlerin tablodaki olasılıklarına bakılır. Örneğin 4/19 olasılıklı bir değer gelirse bu Order 0'da "a"dır. 2/5 olasılıklı bir değer gelirse sembolün geçmişine bakılır; geçmişinde "a" olup Order 1'de bu olasılığa sahip sembol aranır. Escape gelmişse o context'teki sembol değil, bir alt order'a switch edildiği anlaşılır.

7. PPM'in Versiyonları: PPMA, PPMB, PPMC

PPM'in farklı versiyonları vardır. Hoca, insanların sıkıştırma performansını en ufak bir noktada bile artırabilmek için parametrik değerler üzerinde değişiklik yaptığını; PPM'de de escape sembolünün olasılığını belirlerken farklılıklar yapıldığını söyledi. Slayttaki bilgiler (ekrandan, 00:50:00):

- PPM'in üç farklı versiyonu vardır. Bu modellerin yalnız escape sembollerinin kullanımında farklılıklar vardır. Sıkıştırma oranlarında da çok küçük farklar oluşur.
- Bir önceki gördüğümüz yöntem PPMC'dir.
- **PPMA:** toplam frekans n ise escape sembolüne $1/(n+1)$ olasılığı, diğer sembollere de x/n olasılığı atanır.
- **PPMB:** C context'inin içeriğinde s iki kez göründükten sonra C context'ini izleyen S sembolüne bir olasılık atanır. Eğer "abc" context'i üç kez ve bunu izleyen "x" iki kez, "y" de bir kez görünmüşse x için $(2-1)/3$, y için de $(1-1)/3$ olasılığı atanır.

> **Ders sonrası not:** Slayttaki PPMA ifadesinde escape $1/(n+1)$, diğer semboller x/n olarak yazılmıştır. Bu durumda olasılıkların toplamı 1'i aşar; diğer sembollere büyük olasılıkla $x/(n+1)$ atanmalıdır.

Hoca PPMB'yi ekranda "abcx ... abcx ... abcx" dizisiyle açıkladı. "abc" context'i ilk kez göründüğünde ardından gelen "x" için bu context'e olasılık atanmaz; x'e yalnız kendi (daha kısa context'teki) olasılık değeri verilir. Aynı context tekrar görüldükten sonra "x" bu "abc" context'iyle tabloya eklenir; ardından "bc" ve "c" context'leriyle de eklenir. Bu algoritma daha maliyetlidir, çünkü context'lerin en az iki kez geçip geçmediğini tutmak gerekir.

> **Hocanın vurgusu:** En popüler ve en çok kullanılan versiyon PPMC'dir; bu yüzden derste o gösterildi. A ve B'den sonra geldiği için daha başarılı olması beklenir.

8. Pratik Gerçekleştirme: Trie Ağaçları

Slayttaki ifade (ekrandan, 00:50:00): PPM'in pratikteki uygulamasında ana problem, input stream'den okunan her sembolün 0'dan N'ye kadar bütün context'lerinin veri yapısını oluşturmak ve bunlara hızlı bir şekilde erişmektir. Bunun için "trie" adı verilen özel bir veri yapısı kullanılır; ağacın her dalı birbirine yakın context'lerden oluşur.

Hoca, tablo üzerinde gözle aramanın kolay olduğunu, ama kod yazarken bunun mümkün olmadığını anlattı. Örneğin "a"dan önce "im" var mı diye tabloda satır satır bakmak gerekir; bir metinde çok sayıda farklı context çıkacağı için sıralı arama yapılamaz. Bu yüzden trie ağaçları kullanılır. Hoca, trie'nin doğal dil işlemede information retrieval kısmında ve lisans veri yapıları dersinde de geçtiğini, adının "retrieval" kelimesinden geldiğini söyledi.

8.1 "zxzyzxxyzx" için trie (ekrandan, 00:57:30)

Slaytta "zxzyzxxyzx" metni için on trie gösterilir (Salomon, Figure 2.74). Ağaç bir kökle başlar ve kök boştur. Her düğüm "sembol, sayı" çiftidir. Hoca adımları şöyle anlattı:

- **1. z:** Kökün altına yaprak olarak (z,1) eklenir. z şu ana kadar bir kez gelmiştir.
- **2. x:** x yoktur; kökün altına (x,1) eklenir. Sonra geçmişine bakılır: geçmişte z vardır, z'nin altına (x,1) eklenir. "zx" bir kez görülmüştür.
- **3. z:** z ikinci kez gelmiştir, (z,2) olur. Geçmişi x'tir; x'in altına (z,1) eklenir. Order 2'de geçmişi "zx" olan z için zx dalının altına (z,1) eklenir.
- **4. y:** y daha önce gelmemiştir; kökün altına (y,1). Geçmişi z olan y yoktur; z'nin altına (y,1) eklenir. Geçmişi "xz" olan y de yoktur; x→z dalının altına (y,1) eklenir.
- **5. z:** Kökteki z 3 olur. y'nin altına z eklenir (yz). Geçmişi "zy" olan z için z→y dalının altına z eklenir.
- **6-10. x, x, y, z, x:** Aynı şekilde devam edilir. Örneğin x tekrar gelince kökteki x sayısı artar, x'in geçmişindeki context'lerin sayıları da artar; "yz" geçmişli x ikinci kez gelince yz→x 2 olur.

Onuncu adımda kökün altında x 4, y 2, z 4 sayıları vardır. Derinlik 1 Order 0'ı, derinlik 2 Order 1'i, derinlik 3 Order 2'yi gösterir. Her düğümdeki frekanslardan olasılıklar çıkarılabilir; bu ağaca escape kodları da eklenir. Böylece tabloda yapılan işin aynısı ağaçta yapılmış olur ve ağaç üzerinde çok hızlı arama yapılabilir.

9. Vine Pointer ile Switch Etme

Tabloda önce Order 2'ye bakılıyor, bulunamazsa Order 1'e, sonra Order 0'a switch ediliyordu. Bunun ağaçta nasıl yapılacağı slaytta şöyle anlatılır (ekrandan, 01:00:00):

- Algoritmayı basitleştirmek için her bir düğüme bir pointer eklenir.
- Bu pointer geriye doğru bir sonraki kısa context'i gösterir.
- Ağaçta geriyi işaret eden bu pointer "wine pointer" olarak adlandırılır.

> **Ders sonrası not:** Slaytta "wine pointer" yazıyor; literatürdeki adı büyük olasılıkla "vine pointer"dır (Salomon).

Pointer'lar trie oluşturulurken eklenir. Slaytta "assanissimassa"nın ilk altı trie'si gösterilir (Salomon, Figure 2.75). İlk "a" eklendiğinde (a,1) düğümünün pointer'ı köke gösterir. "s" geldiğinde kökün altına (s,1), a'nın altına (s,1) eklenir; "as" düğümünün pointer'ı "s" düğümünü, "s" düğümünün pointer'ı kökü gösterir. İkinci "s" geldiğinde en uzun context'te (as→s) aranır; orada bulunamazsa pointer ile bir kısa context'e (s→s), orada da bulunamazsa tek "s"ye, en sonunda köke gidilir. Köke çıkmak, bir alt modele, yani Order 0'dan sonra yeni sembol durumuna geçmek demektir.

Hoca, tabloda escape ile sağlanan switch etme olayının trie ağacında bu pointer'larla aynen gösterildiğini ve bütün verilerin trie'de toplandığını söyledi. PPM, istatistiksel yöntemlerden derste gösterilen son yöntemdir.

10. Ders İçi Alıştırma: Huffman Kodları ve Aritmetik Kodlama Tablosu

Sözlük tabanlı yöntemlere geçmeden önce hoca, vize öncesi hazırlık gibi iki küçük soru verdi. Önce notlara bakmadan çözmeyi, takılınca notlara bakmayı istedi; amacı nerede takılındığını görmekti.

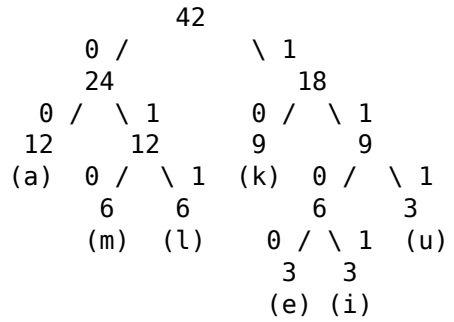
10.1 Huffman kodları (ekrandan, 01:20:00 ve 01:42:50)

Kullanım sıklıkları 3, 3, 3, 6, 6, 9, 12 olan sembollerin Huffman kodları istendi. Ekranda frekansların altında e, i, u, m, l, k, a harfleri yazılıdır (e = 3, i = 3, u = 3, m = 6, l = 6, k = 9, a = 12).

Birleştirme aşaması. Frekanslar zaten küçükten büyüğe sıralıdır. Algoritma, arka arkaya gelen en küçük iki sıklığı toplar ve toplamı sıralamayı bozmayacak şekilde yerleştirir:

- $3 + 3 = 6$. Listede 6 zaten vardır; yeni 6 mevcut 6'ların soluna konur. Liste: 3, 6, 6, 6, 9, 12.
- $3 + 6 = 9$. Yine aynı sayı vardır; tutarlı olmak için yine soluna konur. Liste: 6, 6, 9, 9, 12.
- $6 + 6 = 12$, mevcut 12'nin soluna. Liste: 9, 9, 12, 12.
- $9 + 9 = 18$. Liste: 12, 12, 18.
- $12 + 12 = 24$. Liste: 18, 24.
- $24 + 18 = 42$. Kök 42'dir.

Ağacın kurulması. En son bulunan değer, 42, kök olarak yazılır. Altına toplamları 42 olan iki değer gelir. İlk okunan sola, ikinci okunan sağa konur. Ekranda toplam değerler kırmızı, gerçek frekanslar siyahtır; siyah olan, yani gerçek frekans olan değerler yapraktır.



Sol dallar 0, sağ dallar 1 ile etiketlenir. Kodlar yukarıdan aşağıya okunur:

```
12 (a) -> 00
 6 (m) -> 010
 6 (l) -> 011
 9 (k) -> 10
 3 (u) -> 111
 3 (e) -> 1100
 3 (i) -> 1101
```

Hoca 12'ye hangi harfi verdiğini hatırlamadığını söyleyerek kodları frekanslarla okudu; harf eşlemesi ekrandaki birleştirme sırasından kuruldu (ara adım eklendi).

Prefix kontrolü. Bir sembolün kodu başka bir sembolün kodunun başlangıcı olamaz. `00`, `010`, `10`, `011` ile başlayan başka kod yoktur; bu yüzden kodlar prefix koddur. Ortalama kod uzunluğu derste hesaplanmadı:

> **Çözüm (derste verilmedi):** $\bar{L} = (12\cdot 2 + 6\cdot 3 + 6\cdot 3 + 9\cdot 2 + 3\cdot 3 + 3\cdot 4 + 3\cdot 4)/42 = 111/42 \approx 2{,}64$ bit/sembol. Bu değer standart Huffman birleştirmesiyle bulunan en küçük değerle aynıdır (python ile doğrulandı).

Farklı ağaç da doğrudur (ekrandan, 01:47:20). Bir öğrenci ilk okuduğunu sola yerleştirmediği için hata yaptığını düşündü. Hoca onun ağacını (kökün solunda 18, sağında 24) ekranda kurdu ve bunun da prefix kod verdiğini gösterdi. Kodlar farklıdır ama her sembolün kod uzunluğu aynıdır; bu yüzden entropi, ortalama mesaj uzunluğu ve redundancy değişmez.

> **Hocanın vurgusu:** Toplam, verilmiş frekanslardan biriyle aynıysa ister soluna ister sağına yerleştirilebilir; kağıt üzerinde bunun önemi yoktur. Ama hep sola koyduysanız her adımda sola koymalısınız. Ağaç kurulurken de ilk okunan hep sola (ya da hep sağa) konmalıdır. Her adımda fikir değiştirilirse prefix kod olma özelliği kaybedilebilir.

Hoca, A noktasından B noktasına Huffman ile sıkıştırılmış veri gönderilirken bu yerleştirme kuralının önemli olduğunu da hatırlattı: kodlayıcı ve kod çözücü aynı kuralı kullanmalıdır.

10.2 Aritmetik kodlama tablosu (ekrandan, 01:53:30)

Üç sembol verilir: a 0,4, b 0,3, c 0,29. Aritmetik kodlamaya hazır hale getirmek için her sembolün alt (low) ve üst (high) sınırlarının tablosu istenir.

Hoca önce aritmetik kodlamadaki problemi hatırlattı: semboller $[0, 1)$ aralığına dağıtıldığında alt sınırı 0 olan bir sembol vardır; doküman bu sembolle biterse kod çözmede sorun olur ve bir escape kodu eklenir. Bunun gerekip gerekmediği olasılıkların toplamından anlaşılır: $0{,}4 + 0{,}3 + 0{,}29 = 0{,}99$. Toplam 1 değildir; 1'e tamamlamak için escape'e 0,01 gibi küçük bir olasılık verilir.

Semboller $[0, 1)$ aralığına yukarıdan aşağıya yerleştirilir:

- a: high 1 olsun. Aralığın 0,4 olması için low 0,6'dır: $[0{,}6;\ 1)$.
- b: 0,6'da (açık) biter; 0,3 genişliği için low 0,3'tür: $[0{,}3;\ 0{,}6)$.
- c: 0,3'te biter; 0,29 çıkarılınca low 0,01'dir: $[0{,}01;\ 0{,}3)$.
- esc: $[0;\ 0{,}01)$.

Kod çözerken kümülatif frekansa da ihtiyaç vardır; frekanslar verilmiş olsaydı onlar da bu tablodan çıkarılabilirdi. Hoca ayrıca kod çözmede, formüllerin sonucunda bulunan değerin hangi sembolün aralığında kaldığına bakıldığını hatırlattı.

> **Hocanın vurgusu:** "Gördük, anladık" dediğimiz şeyleri o kadar net anlamamış olabiliriz. Çok fazla yöntem var; arada bir dönüp bakılmazsa iş zorlaşır.

11. Sözlük Tabanlı Yöntemlere Giriş

Bölüm 6 "Sözlük Tabanlı Yöntemler" ile yeni bir aileye geçilir (ekrandan, 01:20:00). Bundan sonraki yöntemlerde istatistik, yani olasılık hiç kullanılmaz; kendi oluşturduğumuz sözlükler kullanılır. Bu sözlük TDK sözlüğü gibi bir sözlük değildir: belki tek karakter sözlükte bir sembol olur, belki 6 karakter bir sembol olur. Bir araya gelen karakterlerin anlamlı bir şey ifade etmesi gerekmez.

11.1 İstatistiksel yöntemlerden farkı

- İstatistiksel sıkıştırma yöntemleri verinin istatistiksel modelini kullanır ve sıkıştırmanın kalitesi seçili modele bağlıdır.
- Sözlük tabanlı yöntemlerde ne istatistiksel modeller ne de değişken uzunluklu kodlar kullanılır. Yani ne olasılık bilgisi ne de "bu sembol 3 bitte, şu sembol 4 bitte kodlanır" gibi değişkenler vardır.

- Sembollerden oluşan stringler ve bunları ifade eden **token**'lar kullanılır. Token bazen iki, bazen üç elemandan oluşur: stringin kendisi, offset (stringin doküman içindeki yeri) ya da kaç karakterin uyuştuğunu gösteren length bilgisi.

11.2 Statik ve dinamik sözlükler

- **Statik sözlük:** Önceden hazırlanmış, sabit sözlüktür. Bazı algoritmalarda sözlüğe yeni string eklenmesine izin verilir, bazılarında verilmez. Bazılarında çok az kullanılan stringler sözlükten silinir; bu seçilen algoritmaya bırakılmıştır.
- **Dinamik sözlük:** Input stream okundukça oluşur. Daha önce bulunmuş stringler tutulur; giriş okunurken ekleme ve silme yapılır.

12. Statik Sözlükle Kodlama ve Sözlükte Bulunma Oranı

12.1 İngilizce sözlük örneği (ekrandan, 02:00:00)

Slayttaki ilkeler:

- Statik sözlüğün en basit örneği İngilizce dokümanları sıkıştırmak için kullanılan İngilizce diline ait sözlüktür. Kelimeler ek aldığı için Türkçe için uygun olmaz.
- Tanımları olmaksızın yarım milyon kelimeden oluşan bir sözlük düşünün.
- Bir kelime input stream'den okunur ve sözlükte aranır. Kelime sözlükte bulunursa sözlükteki sırası ile output stream'e yazılır.
- Kelime sözlükte bulunmazsa sıkıştırılmadan output stream'e aynen yazılır.
- Sözlükte yarım milyon kelime varsa, $2^{19} = 524.288$ kelime için index alanı olarak 19 bite ihtiyaç vardır.
- Output stream, kelimenin sözlükteki sıra değeri (index) ve ham kelimelerden oluşur. Bu iki değeri birbirinden ayırt etmek gerekir; bunun için ekstra bir bit kullanılır.

Hoca, İngilizce ekli bir dil olmadığı için (yalnız birkaç takı vardır) ekleri sözlüğe kelimeyle birlikte ya da ayrı bir index olarak koymanın mümkün olduğunu, ama Türkçede bir kelimeye onlarca ek gelebildiği için bunun uygun olmadığını söyledi. Örneğin kelime sözlükte 752. sıradaysa output'a 752 yazılır. Sözlük on yıl önce hazırlanmışsa "Twitter" gibi sonradan dile giren bir kelime sözlükte yoktur ve harf harf yazılır.

12.2 Kelime başına bit sayısı (ekrandan, 02:10:00)

Karakter uzunluğu 5 olan bir kelime için:

```
Sözlükte var (20 bit):
| flag: 1 bit (0) | index: 19 bit |

Sözlükte yok (48 bit):
| flag: 1 bit (1) | #karakter: 7 bit | 1. harf: 1 byte | 2. harf | 3. harf | 4. harf | 5. harf
```

Flag 0 ise arkadan gelen 19 bit okunur; bu kelimenin sözlükteki sırasını verir ve oradaki kelime alınır. Flag 1 ise kelime sözlükte yoktur; flag ile birlikte bir byte oluşturan sonraki 7 bit kelimenin kaç karakterden oluştuğunu söyler. Örneğin 5 yazıyorsa sonraki 5 byte sırayla okunur. Böylece $1 + 7 + 5 \times 8 = 48$ bit gerekir.

Soru: Kelimenin 5 karakterli olmasının ya da uzunluğun 7 bitle kodlanmasının bir önemi var mıydı? **Cevap:** Gerçek hayatta kelimeler her zaman 5 karakter değildir; uzunluk 7 bitle tutulduğu için en fazla 128 karakter ifade edilebilir. Soruda hesap kolay olsun diye 5 alınmıştır.

> **Ders sonrası not:** Hoca bir yerde "sözlükte varsa 9 bitlik index" dedi; doğrusu 19 bittir. Soruyu tekrar ederken "500 bin kelimeyi sıra numarasıyla kodlamak için 20 bit" demesi, 1 bitlik flag ile birlikte toplamdır; index 19 bittir.

12.3 Kelimelerin ne kadarı sözlükte olmalı? (ekrandan, 02:14:30)

Hoca bu soruyu sınıfa çözdürdü ve bir vizede de sorduğunu söyledi. Bir öğrenci %42 önerdi; hoca birlikte bakalım dedi.

N kelime olsun ve hepsi 5 karakterlik olsun. Sıkıştırmasız kodlamada her kelime $5 \times 8 = 40$ bittir; toplam $40N$ bit. Bir kelimenin sözlükte olma olasılığı P, olmama olasılığı $1 - P$ olsun. Sözlükteki kelimeler 20 bitle, olmayanlar 48 bitle kodlanır:

$$N\,[20P + 48(1 - P)] = 40N$$

N'ler sadeleşir: $20P - 48P + 48 = 40$, yani $-28P + 48 = 40$ ve

$$28P = 8 \;\Rightarrow\; P = \frac{8}{28} \approx 0{,}29$$

$P > 0{,}29$ ise sıkıştırma yapılmış olur. P %29'un altındaysa sıkıştırma değil genişletme yapılır; tam sınırda kodlama aynı uzunlukta kalır. Hoca pratikte %30'dan fazla kelime sözlükteyse sıkıştırma olduğunu söyledi. Python ile $8/28 = 0{,}2857$ bulundu. Öğrencinin önerdiği %42 sınırın üstündedir ama sorunun cevabı olan sınır değil.

Kelime uzunlukları farklı olsaydı 48 ve 40 sayılarıyla bu kadar kolay sınırlandırılamazdı. Slayttaki not: özel durumlarda birkaç yüz kelimeden oluşan statik sözlüklerin kullanılması daha iyi sonuç verecektir. Hoca bunu finans örneğiyle açıkladı: finans metinlerinde geçen kelimeler bellidir, sözlükte hiçbir zaman 500 bin kelime olmaz; kelime sayısı az olduğu için sözlükle çok rahat sıkıştırma yapılır.

LZ77, LZ78 ve LZW gibi yöntemler sonraki derse bırakıldı; bu derste yalnız string sıkıştırmada sözlükle çalışmanın neden kullanışlı olduğu bir örnekle gösterildi.

13. String Sıkıştırma: Neden Karakter Değil String?

Slayttaki ilke (ekrandan, 02:17:40): sembollerden oluşan string'lere dayalı sıkıştırma yöntemleri, tek bir sembole dayalı yöntemlerden daha etkilidir.

13.1 Tek sembol

İki sembollü bir alfabe olsun: $a_1 \to P_1 = 0{,}8$, $a_2 \to P_2 = 0{,}2$.

- Ortalama olasılık değeri: $(0{,}8 + 0{,}2)/2 = 0{,}5$.
- Varyans: $|0{,}8 - 0{,}5| + |0{,}2 - 0{,}5| = 0{,}6$.
- 0,6 ya da 0,5 bit gibi kesirli bit olamayacağı için bu iki sembol 1 bit ile kodlanır (biri 0, diğeri 1; prefix koddur). Kodun ortalama uzunluğu 1 bit/sembol'dür.

  Ek bilgi (derste anlatılmadı): Burada "varyans" diye hesaplanan değer, ortalamadan mutlak sapmaların toplamıdır; istatistikteki varyans (kare sapmaların ortalaması) değildir. Ayrıca $P = (0{,}8;\ 0{,}2)$ kaynağının entropisi yaklaşık 0,722 bit/sembol'dür; string uzadıkça ortalama kod uzunluğu bu sınıra yaklaşır.

13.2 İkili stringler (ekrandan, 02:20:00)

Şimdi a_1 ve a_2 tek tek değil, ikili stringler olarak kodlanır. İki sembolle $2^2 = 4$ farklı durum vardır. Bir stringin gelme olasılığı, sembol olasılıklarının çarpımıdır:

```
String  Probability          Code
a1a1    0.8 x 0.8 = 0.64     0
a1a2    0.8 x 0.2 = 0.16     11
a2a1    0.2 x 0.8 = 0.16     100
a2a2    0.2 x 0.2 = 0.04     101
```

Bu olasılıklarla Huffman ya da Shannon-Fano ile kodlama yapılabilir. Olasılığı yüksek olan a_1a_1 1 bitle, olasılığı düşük olanlar 3 bitle kodlanmıştır.

- Ortalama olasılık: $1/4 = 0{,}25$.
- Varyans: $|0{,}64 - 0{,}25| + |0{,}16 - 0{,}25| + |0{,}16 - 0{,}25| + |0{,}04 - 0{,}25| = 0{,}78$.
- Huffman ortalama kod uzunluğu: $1 \times 0{,}64 + 2 \times 0{,}16 + 3 \times 0{,}16 + 3 \times 0{,}04 = 1{,}56$ bit/string.
- Bir string iki sembol olduğu için $1{,}56/2 = 0{,}78$ bit/sembol.

Tek tek kodlarken bir sembol 1 bitle kodlanıyordu; ikili kodlamada bir sembol 0,78 bite düşer. Hoca bunun PPM'deki gibi olduğunu söyledi: tek karakter yerine sembol dizisi kodlanınca başarım artar.

> **Ders sonrası not:** Hoca "iki sembolden 2 üzeri 4" dedi (ekrana da 2^4 yazdı); ikili stringlerin sayısı $2^2 = 4$'tür. Hocanın ortalama uzunlukta "0,14" ve "2,56" dediği yerler slaytta $3 \times 0{,}04$ ve 1,56'dır. Bütün değerler python ile doğrulandı.

13.3 Üçlü stringler (ekrandan, 02:24:00)

Üç sembolden $2^3 = 8$ farklı durum vardır. Hoca bunları ikilik sayılar gibi düşünmeyi önerdi: 0'lar a_1'i, 1'ler a_2'yi temsil eder.

String	Probability	Code
a1a1a1	0.8 x 0.8 x 0.8 = 0.512	0
a1a1a2	0.8 x 0.8 x 0.2 = 0.128	100
a1a2a1	0.8 x 0.2 x 0.8 = 0.128	101
a1a2a2	0.8 x 0.2 x 0.2 = 0.032	11100
a2a1a1	0.2 x 0.8 x 0.8 = 0.128	110
a2a1a2	0.2 x 0.8 x 0.2 = 0.032	11101
a2a2a1	0.2 x 0.2 x 0.8 = 0.032	11110
a2a2a2	0.2 x 0.2 x 0.2 = 0.008	11111

En büyük olasılık üç a_1'in arka arkaya gelmesi, en küçük olasılık üç a_2'nin arka arkaya gelmesidir.

- Ortalama olasılık: $1/8 = 0{,}125$.
- Varyans: $|0{,}512 - 0{,}125| + 3|0{,}128 - 0{,}125| + 3|0{,}032 - 0{,}125| + |0{,}008 - 0{,}125| = 0{,}792$. Aynı değerler tekrar ettiği için 3 ile çarpılmıştır.
- Ortalama Huffman kod uzunluğu: $1 \times 0{,}512 + 3 \times 3 \times 0{,}128 + 3 \times 5 \times 0{,}032 + 5 \times 0{,}008 = 2{,}184$ bit/string.
- $2{,}184/3 = 0{,}728$ bit/sembol.

> **Ders sonrası not:** Hoca kodların "1, 2, 4, 5 bit" olduğunu söyledi; tabloda kod uzunlukları 1, 3 ve 5 bittir. Slaytta "2.1846/3" yazıyor; doğru değer 2,184'tür ve sonuç 0,728 değişmez (python ile doğrulandı).

13.4 Özet tablo

Str. size	Variance of prob.	Avg. size of code
1	0.6	1
2	0.78	0.78
3	0.792	0.728

Tek sembol kodlanırsa ortalama kod uzunluğu 1, ikili kodlanırsa 0,78, üçlü kodlanırsa 0,728 olur; 4'lü, 5'li kodlandıkça bu değer azalır ve daha ideal bir kodlamaya yaklaşılır. Varyans da gittikçe yükselir. Bu değerler iki sembolden türetildiği için birbirine yakındır; ama 29 karakterlik bir alfabenin üçlü kombinasyonları düşünülürse çok daha fazla farklı string vardır ve olasılıkları birbirine bu kadar yakın olmaz.

> **Hocanın vurgusu:** Karakter bazında çalışmayın; çalışacaksanız daha büyük stringler bazında çalışın.

13.5 Türkçe için heceler ve kök-ek ayrımı

Hoca bu fikrin Türkçeye uygulanmasıyla ilgili örnekler verdi:

- **Kök ve eklerle kodlama:** Metin morfolojik olarak çözümlenir ve morfolojik birimler sembol olarak alınır. Kökler ve ekler ayrı ayrı Huffman ağaçlarında kodlanırsa performans çok daha yükselir; ancak kod çözerken hangi ağaçtan çözüleceğinin bilinmesi gerekir. Ayrıca ünsüz değişmesi gibi durumlar vardır: metinde "kitabı" yazar, çözümlemede "kitap + ı" bulunur; açarken bunun yeniden "kitabı"ya dönüştürülmesi gerekir.

- **Hecelerle kodlama:** Metin hecelere ayrılır ve heceler string olarak kodlanır. Türkçede 4 karakterden oluşan çok az hece vardır (örneğin "Türk"); hecelerin çoğu 2 ya da 3 karakterlidir, tek karakterli hece de azdır. Tek karakterli bir hece ardından gelen boşlukla birlikte bir sembol olarak alınabilir.

Amaç şudur: ne kadar çok karakterden oluşan stringler kodlanırsa o kadar iyi performans elde edilir. Sonraki derste sözlük yöntemlerinden LZ77, LZ78 ve bunların en iyi performans veren türevi LZW anlatılacak, token kavramına geri dönülecektir.

Örnek Sorular

Soru 1 (ekrandan, 00:42:20): "assanissimassa" tablosuyla (Order 2, PPMC) son context "sa" iken sıradaki sembol sırasıyla n, s, m ve d ise her biri kaç bitle kodlanır?

Çözüm (derste verildi): n → sa→n 1/2, 1 bit. s → esc 1/2 (1 bit) + a→s 2/5 (1,32 bit) = 2,32 bit. m → 1 + 1,32 + $-\log_2(1/19)$ = 4,25 → 6,57 bit. d → 1 + 1,32 + $-\log_2(5/19)$ = 1,93 + $-\log_2(1/28)$ = 4,8 → 9,05 bit.

Soru 2 (ekrandan, 01:20:00): Kullanım sıklıkları 3, 3, 3, 6, 6, 9, 12 olan sembollerin Huffman kodlarını bulun.

Çözüm (derste verildi): Birleştirme: 3+3 = 6, 3+6 = 9, 6+6 = 12, 9+9 = 18, 12+12 = 24, 24+18 = 42. Kodlar: 12 → `00`, 6 → `010`, 6 → `011`, 9 → `10`, 3 → `111`, 3 → `1100`, 3 → `1101`. Kodlar prefix koddur.

> **Çözüm (derste verilmedi):** Ortalama kod uzunluğu $111/42 \approx 2{,}64$ bit/sembol (python ile doğrulandı).

Soru 3 (ekrandan, 01:53:30): Olasılıkları a 0,4, b 0,3, c 0,29 olan semboller için aritmetik kodlamanın başlangıç tablosunu (low ve high sınırları) oluşturun.

Çözüm (derste verildi): Toplam 0,99 olduğu için eksik 0,01 escape sembolüne verilir. a $[0{,}6;\ 1)$, b $[0{,}3;\ 0{,}6)$, c $[0{,}01;\ 0{,}3)$, esc $[0;\ 0{,}01)$.

Soru 4 (ekrandan, 02:14:30): 500.000 kelimelik statik sözlükte bulunan kelime 20 bit, bulunmayan 5 karakterlik kelime 48 bit ile kodlanıyor. Kelimelerin ne kadarı sözlükte bulunmalıdır ki sıkıştırmadan bahsedilebilsin?

Çözüm (derste verildi): $N[20P + 48(1-P)] = 40N \Rightarrow 28P = 8 \Rightarrow P \approx 0{,}29$. $P > 0{,}29$ olmalıdır; altında genişletme olur.

Soru 5 (ekrandan, 02:20:00 ve 02:24:00): $P(a_1) = 0{,}8$, $P(a_2) = 0{,}2$ olan bir alfabede tek, ikili ve üçlü string'lerle Huffman kodlamada sembol başına ortalama kod uzunluğu nedir?

Çözüm (derste verildi): Tek sembol 1 bit; ikili 1,56/2 = 0,78 bit; üçlü 2,184/3 = 0,728 bit/sembol.

Dersin Toparlaması

Bu derste PPM tamamlandı. PPM, sembolün önündeki N sembolü context olarak kullanır ve olasılıkları aritmetik kodlamaya gönderir. Uzun context'te sembol bulunamazsa daha kısa context'e switch edilir ve her switch'ten önce o context'in escape olasılığı gönderilir; Order 0'da da bulunamayan yeni sembole $1/$alfabe boyu verilir. "assanissimassa" örneği, bir sembolün context'e göre 1 bitten 9 bite kadar farklı uzunluklarda kodlanabildiğini gösterdi. PPMA, PPMB ve PPMC yalnız escape'in kullanımında ayrılır; pratikte context'ler trie ağacında tutulur ve vine pointer'lar switch etmeyi ağaç üzerinde sağlar.

Ders içi alıştırmada Huffman ağacında yerleştirme tutarlılığının ve aritmetik kodlamada olasılık toplamının 1'e tamamlanmasının önemi tekrar edildi. Ardından sözlük tabanlı yöntemlere geçildi: bu yöntemler olasılık ve değişken uzunluklu kod kullanmaz, string ve token'larla çalışır. Statik sözlük örneği, kelimelerin yaklaşık %29'undan fazlası sözlükte olmazsa sıkıştırma yapılamayacağını gösterdi. String sıkıştırma örneği de string boyu arttıkça sembol başına kod uzunluğunun düştüğünü gösterdi. Sonraki derste LZ77, LZ78 ve LZW anlatılacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.